

DOCUMENTATION TECHNIQUE

Service d'intelligence artificielle locale

FastAPI : détection d'anomalies par Isolation Forest, vision YOLOv8 et ByteTrack, reconnaissance faciale YuNet et SFace, fusion de capteurs et moteur de risque.

Projet	Sentinel-X — Workshop EPSI Bac+4, consortium AetherCorp
Pile	Python 3.12, FastAPI, Uvicorn, scikit-learn, NumPy, OpenCV, Ultralytics YOLOv8, Pydantic
Public	Équipe IA et data, développeurs et jury technique
Version	Octobre 2026 — état du dépôt à la date de génération

Sommaire

- 01** Rôle et intégration
Position du service, principes

- 02** Organisation du code
Modules et responsabilités

- 03** Détection d'anomalies
Features, entraînement, inférence, scores

- 04** Vision par ordinateur
Capture, YOLOv8, ByteTrack, MJPEG

- 05** Reconnaissance faciale
YuNet, SFace, catalogue, enrôlement

- 06** Fusion et moteur de risque
Règles, pondérations, niveaux

- 07** API
Routes et format de prédiction

- 08** Configuration
Variables d'environnement

- 09** Sécurité, robustesse, performances
Jeton, pannes, ressources

- 10** Tests et limites
Couverture, résultats, précautions

1 Rôle et intégration

Le service IA s'exécute localement sur le PC Serveur Local, par défaut sur `127.0.0.1:8001`. Il n'est jamais appelé par le navigateur : le backend lui transmet chaque mesure récente par HTTP, persiste les résultats dans SQLite et les diffuse au dashboard par SSE. La webcam est branchée sur le serveur ; le service l'analyse et produit un flux MJPEG annoté, relayé par le backend derrière la session.

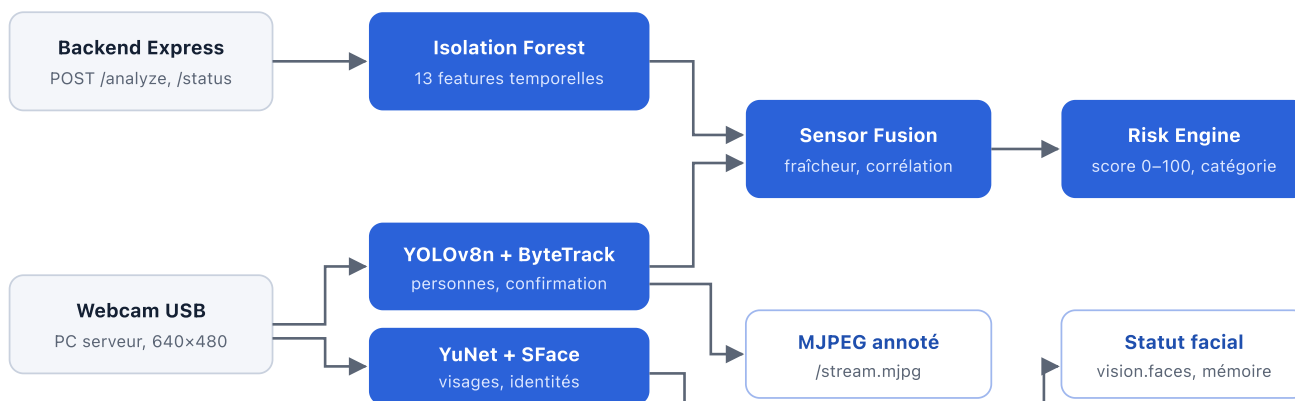


Figure 1. Chaîne d'analyse du service IA. Le backend reçoit le résultat de la fusion et du moteur de risque.

À SAVOIR

Principes : aucune prédiction inventée (les inconnues valent `null`), aucun modèle de production fourni ni entraîné automatiquement, séparation stricte des données réelles et simulées, et une panne de l'IA n'interrompt jamais l'ingestion des capteurs.

2 Organisation du code

Module (ai/app/)	Responsabilité
<code>main.py</code>	Application FastAPI, routes, authentification par jeton, démarrage Uvicorn avec réservation du port
<code>config.py</code>	Lecture et validation des variables d'environnement ; chemins relatifs résolus depuis <code>ai/</code>
<code>service.py</code>	Orchestration : analyse d'une mesure, état courant, rechargement du modèle
<code>schemas/</code>	Modèles Pydantic : mesures, prédictions, vision, visages
<code>anomaly/</code>	<code>preprocessing.py</code> (features), <code>trainer.py</code> (entraînement CLI), <code>model_manager.py</code> (artefact), <code>detector.py</code> (inférence)
<code>vision/</code>	<code>camera.py</code> (capture et workers), <code>person_detector.py</code> (YOLO), <code>tracker.py</code> (ByteTrack), <code>face_recognition.py</code> , <code>face_enrollment.py</code> , <code>setup_faces.py</code> (téléchargement vérifié des modèles)
<code>fusion/</code>	<code>sensor_fusion.py</code> (preuves, fraîcheur) et <code>risk_engine.py</code> (score et catégorie)

Module (ai/app/)	Responsabilité
integrations/backend_client.py	Export CSV des mesures réelles depuis la base SQLite du backend (lecture seule)
simulator.py	Données synthétiques d'entraînement et démonstration de dérive (commandes explicites)
utils/	Journalisation, santé, détection d'une instance déjà démarrée

3 Détection d'anomalies

3.1 Features

Le même extracteur (`FeatureExtractor`) sert à l'entraînement et à l'inférence. Il produit 13 features à partir d'une fenêtre glissante de 12 mesures (`ANOMALY_WINDOW`) :

Famille	Features
Valeurs brutes	temperature, humidity, gas, presence
Variations	temp_delta, gas_delta, temp_rate, gas_rate
Statistiques glissantes	temp_mean, temp_std, gas_mean, gas_std
Interaction	temp_gas_product

- Les champs manquants sont imputés avec les **médianes apprises**, signalés dans `missing_fields`, et la confiance est réduite. L'imputation ne fabrique aucune hausse artificielle.
- Les séquences sont remises à zéro après une coupure ou un changement de source. Les mesures réelles et simulées utilisent des fenêtres distinctes.
- Les canaux hors limites (−40...80 °C, 0...100 %, gaz 0...1023) sont ignorés individuellement, avec une raison explicite ; les autres restent analysables.

3.2 Entraînement

```
# Export des mesures réelles (source = live) depuis le backend, en lecture seule
python -m app.integrations.backend_client --database ../iot-backend/data/telemetry.db --output
data/sensor_data.csv
# Après sélection des périodes normales :
python -m app.anomaly.trainer --csv data/normal_validated.csv --output
models/isolation_forest.joblib
```

- Format CSV : `timestamp,temperature,humidity,gas,presence`. Les lignes sont triées chronologiquement ; sans horodatage, une cadence de 2 s est supposée.
- Au moins **128 valeurs observées par capteur** sont exigées ; plusieurs milliers de mesures représentatives sont recommandées.
- Modèle : `IsolationForest(n_estimators=200, contamination=0.03, random_state=42)`.

- L'artefact joblib contient le modèle, les médianes, la liste des features, la version de format et la version de scikit-learn. Il est écrit de façon atomique ; un artefact incompatible est refusé au chargement, ce qui impose un réentraînement.
- L'export exclut les mesures simulées et, par défaut, l'historique sans provenance connue (`--include-legacy` pour l'inclure en connaissance de cause), ainsi que les canaux marqués invalides ou en préchauffage par le firmware.

3.3 Résultat

Champ	Signification
<code>status</code>	<code>ready</code> , ou état explicite (modèle absent : <code>untrained</code>)
<code>is_anomaly</code>	Décision de la frontière apprise par l'Isolation Forest
<code>anomaly_score</code>	Distance à la frontière normalisée dans $[0, 1]$ — pas une probabilité
<code>confidence</code>	Marge de décision et disponibilité des données
<code>reason, features</code>	Principaux écarts des features (explication descriptive, non causale)

4 Vision par ordinateur

- **Capture** : index configurable (0 par défaut, -1 pour balayer), DirectShow en priorité sous Windows puis backend OpenCV par défaut. Images 640×480, au plus 15 i/s. L'état passe à `running` uniquement après la première image JPEG disponible.
- **Workers séparés** : capture, YOLO et reconnaissance faciale tournent dans des threads distincts. Un modèle absent, lent ou en erreur laisse la vidéo brute disponible (`detection_status` et `detection_error` explicites).
- **Détection** : YOLOv8n, classe personne uniquement, seuil de confiance 0,55, une image sur trois (`VISION_FRAME_STRIDE`).
- **Confirmation** : une personne est confirmée après 3 analyses consécutives positives. ByteTrack attribue des identifiants de suivi lorsqu'il est disponible ; un échec du tracker conserve la détection.
- **Diffusion** : les rectangles sont dessinés côté serveur ; un seul JPEG récent est partagé entre tous les spectateurs (une seule inférence quel que soit leur nombre). Aucune image n'est écrite sur disque.
- **Cycle de vie** : `start()` et `stop()` sont idempotents. L'arrêt libère la webcam, et une ancienne session ne peut plus publier de résultat.

5 Reconnaissance faciale

Module indépendant : YuNet détecte les visages sur l'image complète, puis SFace calcule des embeddings, comparés par similarité cosinus au catalogue `known_faces/Nom/*.jpg` chargé en mémoire.

Paramètre	Valeur par défaut
Seuil d'identité (similarité)	0,55 — meilleur score parmi les photos d'une identité

Paramètre	Valeur par défaut
Cadence	une image sur 3, 2 analyses par seconde au plus
Taille minimale d'un visage	40 pixels
Historique	20 événements en mémoire, anti-répétition de 5 s par identité ou suivi
Association au suivi	identifiant ByteTrack si le visage est contenu dans une boîte de personne de moins d'une seconde

- **Modèles** : `face_detection_yunet_2023mar.onnx` et `face_recognition_sface_2021dec.onnx`, depuis OpenCV Zoo. `python -m app.vision.setup_faces` les télécharge une fois et vérifie leur empreinte SHA-256. Aucun téléchargement n'a lieu au démarrage.
- **Enrôlement** : 1 à 5 photos (JPEG/PNG/WebP, 5 Mio chacune, 15 Mio au total, 4 096 px et 16 Mpx au plus). Le contenu réel est vérifié, l'orientation EXIF corrigée et les métadonnées supprimées (dont la localisation). Les photos sont réduites à 1 600 px et enregistrées en JPEG nommés par UUID. Une photo sans visage, avec plusieurs visages ou trop petite est refusée avec un motif. En cas d'échec, les fichiers de l'envoi sont retirés.
- **Rechargement** : remplacement atomique du catalogue ; en cas d'échec global, le dernier catalogue valide est conservé.
- Les noms et embeddings ne participent pas au moteur de risque et ne sont pas persistés dans les prédictions SQLite.

6 Fusion et moteur de risque

La fusion combine trois preuves, sous conditions de fraîcheur : capteurs de moins de 30 s, vision de moins de 5 s, tolérance de 5 s dans le futur. Les mesures simulées ne sont jamais corrélées à la webcam réelle.

Preuve d'intrusion	Score d'intrusion	Confiance
PIR actif et personne confirmée par la vision	$80 + 15 \times \text{confiance vision}$	$0,5 + 0,5 \times \text{confiance vision}$
Personne confirmée par la vision seule	$55 + 20 \times \text{confiance vision}$	$0,85 \times \text{confiance vision}$
PIR seul	35	0,35

Score environnemental : uniquement si l'Isolation Forest conclut à une anomalie. Il vaut $45 + 40 \times \text{anomaly_score}$, plus 10 si la température et le gaz augmentent ensemble (corrélations détectées par les features temporelles).

Risque final : le maximum des deux scores. Si intrusion et anomalie environnementale coexistent, la catégorie est `MULTI_THREAT` et le score est majoré de 12 (plafonné à 100). Sinon, la catégorie est `INTRUSION`, `ENVIRONMENT` (anomalie corrélée), `SENSOR_ANOMALY` ou `SAFE`.

Score	0–24	25–49	50–74	75–89	90–100
Niveau	SAFE	LOW	MEDIUM	HIGH	CRITICAL

Si une modalité manque ou est périmée, `degraded=true` et les raisons l'indiquent. Le dashboard affiche alors INDÉTERMINÉ plutôt que SAFE. Les pondérations sont **explicites et démonstratives** : elles ne constituent ni une probabilité calibrée ni une certification de sécurité.

7 API

Route	Rôle
GET /, GET /health	Identification et santé (publiques)
GET /status	Modèle, vision, visages et dernière prédiction
POST /analyze	Anomalie + vision courante + fusion + risque pour une mesure
POST /predict/anomaly	Résultat d'anomalie seul
POST /model/reload	Rechargement du modèle sans redémarrage
GET /vision/status, POST /vision/start, POST /vision/stop	Pilotage et état de la webcam
GET /vision/faces/status, /latest, /history	Catalogue et reconnaissances
POST /vision/faces/reload, POST /vision/faces/enroll	Rechargement et ajout de références
GET /stream.mjpg	Flux MJPEG annoté
GET /risk/latest	Dernier risque, ou null

Entrée de /analyze : {temperature?, humidity?, gas?, presence?, timestamp, sample_id, source}.
Sortie (format illustratif) :

```
{
  "sample_id": 42, "sensor_timestamp": "2026-10-06T16:00:00Z", "source": "live",
  "anomaly": {
    "status": "ready", "is_anomaly": true, "anomaly_score": 0.82, "confidence": 0.64,
    "reason": "Abnormal temperature/gas evolution (correlated rise)",
    "missing_fields": []
  },
  "vision": {
    "status": "running", "person_detected": true, "person_count": 1,
    "max_confidence": 0.91, "confirmed": true, "fps": 14.2, "inference_time_ms": 83.2
  },
  "risk": {
    "risk_score": 100, "risk_level": "CRITICAL", "category": "MULTI_THREAT",
    "reasons": ["Person detected by camera (confirmed)", "PIR sensor active"],
    "confidence": 0.955, "degraded": false
  }
}
```

8 Configuration

Variables lues dans `ai/.env` (modèle : `ai/.env.example`). Une modification exige un redémarrage du service.

Variable	Défaut	Rôle
<code>AI_HOST</code> / <code>AI_PORT</code>	127.0.0.1 / 8001	Adresse d'écoute (locale uniquement)
<code>AI_SERVICE_TOKEN</code> / <code>AI_SERVICE_TOKEN_FILE</code>	vide	Jeton partagé avec le backend (le fichier est prioritaire)
<code>AI_TLS_CERT_FILE</code> / <code>AI_TLS_KEY_FILE</code> / <code>AI_TLS_CA_FILE</code>	vide	Servir en HTTPS (production) ; CA pour le contrôle d'instance
<code>ANOMALY_MODEL</code>	<code>models/isolation_forest.joblib</code>	Artefact du modèle
<code>ANOMALY_WINDOW</code> / <code>ANOMALY_CONTAMINATION</code>	12 / 0.03	Fenêtre des features, contamination à l'entraînement
<code>SENSOR_STALE_SECONDS</code> / <code>VISION_STALE_SECONDS</code>	30 / 5	Fraîcheur des preuves
<code>VISION_CAMERA_INDEX</code> , <code>CAMERA_SCAN_LIMIT</code>	0, 4	Choix de la webcam
<code>YOLO_MODEL</code> , <code>YOLO_CONFIDENCE</code>	<code>models/yolov8n.pt</code> , 0.55	Détecteur de personnes
<code>VISION_CONFIRM_FRAMES</code> , <code>VISION_FRAME_STRIDE</code> , <code>VISION_MAX_FPS</code>	3, 3, 15	Confirmation, cadence d'inférence, capture
<code>VISION_TRACKING</code> / <code>VISION_AUTO_START</code>	true / false	ByteTrack, démarrage automatique de la webcam
<code>FACE_RECOGNITION_ENABLED</code> , <code>FACE_RECOGNITION_THRESHOLD</code>	true, 0.55	Reconnaissance faciale
<code>FACE_RECOGNITION_EVERY_N_FRAMES</code> , <code>FACE_RECOGNITION_MAX_FPS</code>	3, 2	Cadence faciale
<code>FACE_KNOWN_DIR</code> , <code>FACE_MIN_SIZE_PIXELS</code> , <code>FACE_EVENT_COOLDOWN_SECONDS</code>	<code>known_faces</code> , 40, 5	Catalogue et filtrage
<code>FACE_DETECTOR_MODEL</code> , <code>FACE_EMBEDDING_MODEL</code>	<code>models/*.onnx</code>	Modèles YuNet et SFace

9 Sécurité, robustesse, performances

- **Exposition** : écoute sur la boucle locale en développement. En production Docker, le service écoute en HTTPS sur le réseau interne `core` (sans accès Internet, non publié), avec un jeton généré au déploiement. Si un jeton est défini, toutes les routes sauf `/` et `/health` exigent `Authorization: Bearer`. Le jeton reste côté serveur.

- **Image Docker** : PyTorch en version CPU, poids YOLOv8n vérifiés par empreinte SHA-256 et modèles YuNet/SFace intégrés à la construction ; utilisateur non-root, système de fichiers en lecture seule ; modèle entraîné et catalogue facial dans des volumes ; webcam accessible par règle de périphériques vidéo uniquement.
- **Artefacts** : joblib et YOLO désérialisent du code ; ne charger que des fichiers locaux de confiance. Le service n'installe ni ne télécharge rien en production.
- **Démarrage** : le port est réservé avant le chargement des modèles. Une instance Sentinel-X déjà active est détectée (message, code 0, pas de second accès à la webcam) ; un port occupé par un autre service produit une erreur explicite.
- **Pannes** : modèle absent (résultats `null`, statut explicite), webcam absente ou retirée (`unavailable`), YOLO absent (vidéo brute). Le backend continue de fonctionner sans le service IA.
- **Ressources** : une seule capture et un seul JPEG partagé, inférences espacées (1 image sur 3, faces 2/s au plus) ; FPS et temps d'inférence exposés dans le statut.
- **Données personnelles** : photos de référence et modèles ignorés par Git ; aucune image webcam enregistrée ni transmise au navigateur hors du flux vidéo.

10 Tests et limites

Les tests (`pytest`) entraînent leurs propres modèles temporaires sur des données synthétiques et utilisent des doubles pour les caméras et l'inférence. Ils couvrent le preprocessing commun, les données manquantes et invalides, les schémas, la fusion et le risque, la fraîcheur, la santé et le jeton, l'absence de caméra ou de modèle, la confirmation multi-images, le démarrage, l'arrêt et la reprise. Ils couvrent aussi la reconnaissance et l'enrôlement facial (refus, limites, retour arrière), l'export SQLite et la détection d'instance au démarrage.

Vérification (état du dépôt à la génération)	Résultat
<code>python -m pytest -q</code> (environnement de CI : sans modèles, OpenCV headless)	124 réussis — 1 avertissement de dépréciation Starlette

Des scripts d'intégration (`scripts/check-ai-integration.mjs`, `check-face-enrollment.mjs`, `check-camera-integration.mjs`, `check-face-models.py`) lancent des processus réels isolés. Le job *ai* de la CI exécute `pytest` sans modèle, webcam ni secret.

ATTENTION

Limites : aucun modèle de production n'est fourni ; la performance d'un modèle entraîné sur une simulation ne vaut pas validation sur du matériel réel. Les faux positifs de YOLO et le seuil facial doivent être évalués sur des séquences représentatives du site. La reconnaissance faciale est une fonction de démonstration et de monitoring, à ne jamais utiliser comme unique mécanisme de sécurité.